Applied Machine Learning Systems: ELEC0134 (2025/26)

Student Number: SN24205421
Code Repository: https://github.com/KamalJain777/AMLS_25_26_SN24205421

Abstract

This report compares two machine learning methods on the BreastMNIST medical image dataset to analyze the impact of capacity, data augmentation, and training budget have a significant effect on the performance of classification tasks in medical imaging. For Model A (Classical ML) - RBF-kernel SVM is trained on three feature representations including raw flattened pixels, PCA features, and Histogram of Oriented Gradient descriptors (HOG) with hyperparameters chosen via cross-validated grid search. For Model B (Deep Learning), we use a ResNet with a conventional optimisation process with early stopping, and we analyze the effect of augmentation by examining training with and without semantically valid transformations. Both models are tested on a held-out test set using accuracy, precision, recall, F1 score, and ROC-AUC. The results show since Model B outperforms the others in terms of performance (accuracy: 82.69%, AUC: 0.914) in comparison to Model A (accuracy of 79.49% and AUC of 0.844), with data augmentation significantly improving AUC of ResNet. This controlled comparison illustrates the trade-offs between and current end-to-end learning methods in medical image classification under various computational and data limitations [1, 2].
Index Terms— Support Vector Machines, ResNet, Neural Networks, Data augmentation, Medical image classification, BreastMNIST

1 Introduction

Medical image classification is a significant application area of machine learning, wherein the predictions of the images can help clinicians make decisions rapidly. Yet, the performance of the models is affected by several factors, such as the amount of training data, the complexity of the model, as well as the techniques of data augmentation [3]. This report provides a comparison of the performance of several machine learning models on the BreastMNIST dataset, a standard benchmark dataset of grayscale images (28×28 pixels) with binary classification labels (benign versus malignant) [1].

The study uses two representative paradigms:

- **Model A (Classical ML):** A feature-driven pipeline that integrates three representation methods (raw pixels, PCA, Histogram of Oriented Gradients) with an RBF SVM trained using cross-validation and fine tuning via grid search.
- **Model B (Deep Learning):** An end-to-end neural network with residual connections (ResNet model) that learns directly from pixel intensities with possible augmentation.

For both approaches, results are reported in terms of traditional classification evaluation metrics (accuracy, precision, recall, F1-score, confusion matrices and ROC-AUC. This report also enables a systematic comparison between traditional systems and deep learning systems based on capacity, augmentation, and budget, while retaining a constant dataset. This is important in terms of illustrating the trade-offs encountered in applying machine learning approaches to medical imaging problems. [2, 4].

2 Literature Survey

A traditional area in medical image classification is to learn discriminative patterns in images while being invariant to variations in acquisition and noise with a limited number of labelled samples. Prior to the deep learning era, traditional pipelines were predominant in this area: pre-processing was done to transform images to hand-designed descriptors and a traditional classifier was trained on top of them. While deep learning models are known to provide higher accuracy by learning end-to-end features, traditional methods are competitive in the small sample regime [3, 4].

2.1 Classical Machine Learning Approaches

Historical computer vision research has shown that engineered features are efficient at encoding relevant structure. Histogram of Oriented Gradients (HOG) [5] detects local edge orientations and is still a competitive feature on small gray images because it is very informative and geometric variations are unimportant. HOG features combined with a linear or kernel classifier have shown good performance on medical imaging [5].

Principal Component Analysis (PCA) is a well-known technique for dimensionality reduction, especially in high-

dimensional spaces [6]. PCA reduces the effects of the curse of dimensionality by retaining the maximum amount of information by projecting the data into a low-dimensional space [6].

Kernel SVMs have been proven to possess sound mathematical theory and performance capabilities even when operating in high-dimensional spaces [7]. SVM with the radial basis function kernel is capable of handling non-linear classification boundaries with controllable capacity via hyperparameters (C and γ), thus being applicable for medical imaging with small-scale datasets [7].

2.2 Deep Learning in Medical Imaging

The use of Neural Networks brought about a revolution in the direction of end-to-end learning, in which the learning of both feature extraction and classification is performed simultaneously [3]. Residual Networks, or ResNets, solved the issue of vanishing gradients in deep learning models by using skip connections that enabled the direct flow of gradients through the network [8]. ResNet-50, with 50 layers that are grouped into residual blocks with identity skip connections, has been the norm for medical image classification. It allows for the learning of much deeper models without any loss of accuracy, learning more complex representations for better diagnostic results. We chose to implement the ResNet architectural pattern over other architectures such as VGG and DenseNet because: (i) skip connections allow the training process to be stable when the training data is small, by mitigating vanishing gradients, (ii) the compact design prevents overfitting on BreastMNIST's limited 546 training samples, (iii) residual design has demonstrated superior performance in medical imaging benchmarks [4].

3 Description of Models

3.1 Model A: Classical Machine Learning (Feature-Based SVM)

3.1.1 Pipeline Overview

Model A is based on a feature engineering with a classical classifier pipeline. It is suitable for small-sized medical image datasets as it breaks down a learning problem by training a regularized classifier in controlled capacity.

Below figure depicts high level pipeline-

Model A Pipeline (Classical ML)

Raw image (28×28) $\rightarrow$ preprocessing $\rightarrow$ feature extraction (RAW / PCA / HOG) $\rightarrow$ SVM (RBF) with GridSearchCV $\rightarrow$ prediction

Figure 1: Model A pipeline: Classical machine learning approach with feature engineering and RBF SVM classifier

Classifier Choice: An SVM with RBF kernel provides non-linear decision boundary in the feature space while providing flexibility to maintain controlled complexity. The decision function for this SVM is:

$$f(\mathbf{x}) = \text{sign}\left(\sum_{i=1}^{N} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b\right)$$

where the RBF kernel is:

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$$

and C controls the margin-misclassification trade-off and γ defines the kernel width and thus the smoothness of the model. The hyperparameters are chosen through cross-validated grid search to avoid manual tuning of the model and provide a fair comparison across feature representations.

3.1.2 Discussion of Selected Feature Pipelines

Model A benchmarks three complementary feature pipelines under the same classifier:

1. **RAW (Flattened Pixels):** Flattens each 28×28 image into a 784-dimensional vector—provides a simple baseline with minimal assumptions.

2. **PCA Features:** PCA transforms the original features from 784 dimensions to $d \ll 784$ components, thereby reducing the noise and collinearity and generalizing well, especially for small samples [6].

3. **HOG (Histogram of Oriented Gradients):** It calculates the histograms of gradient orientations for local cells or blocks, which are robust to small intensity variations [5].

These three types of representations cover raw data without processing, data compression through statistics, and feature engineering.

3.2 Model B: Deep Learning with Residual CNN

Model B is an end-to-end neural network, which is a small ResNet-style architecture. Model B differs from Model A in the way it trains the feature representation as well as the classifier on the image pixels. This is well suited to image data because convolutions exploit spatial locality and weight sharing, and residual connections improve optimization stability.

3.2.1 Pipeline Overview

Model B implements end-to-end learning where both feature extraction and classification are learned from pixels. Below figure depicts high level pipeline.

Model B Pipeline (Deep Learning) Raw image ($1 \times 28 \times 28$) → (optional) augmentation → tensor normalisation → ResNet-style NN → logits → softmax → prediction

Figure 2: Model B pipeline: End-to-end deep learning approach with ResNet-style Neural Network

3.2.2 Network Architecture and Residual Blocks

A compact version of the ResNet architecture employs residual blocks with skip connections:

$$\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x}; \theta)$$

Skip connections facilitate the flow of the gradient and ensure the stability of the training process. The network accepts $1 \times 28 \times 28$ images and applies convolutional layers, residual blocks, and a two-class classifier outputting logits $\mathbf{z} \in \mathrm{R}^2$. The output probabilities are calculated by the softmax function:

$$p(y = k|\mathbf{x}) = \frac{\exp(z_k)}{\sum_j \exp(z_j)}$$

Training minimizes cross-entropy loss:

$$\mathcal{L} = -\sum_k y_k \log p(y = k|\mathbf{x})$$

A compact ResNet-style neural network offers a good trade-off between capacity and stability: it is expressive enough to learn discriminative patterns from images.

3.2.3 Discussion of Selected Feature Pipelines

Unlike Model A, Model B does not use hand-crafted features such as PCA or HOG. Instead, the feature pipeline is learned inside the neural network during training:

- **Input representation:** the image is treated as a $1 \times 28 \times 28$ tensor.
- **Early layers** learn low-level features (edges, blobs).
- **Deeper layers** learn higher-level patterns by composing earlier features.

In this sense, the "feature extraction pipeline" for Model B is the sequence of convolutional layers and residual blocks which maps iomage pixels to a latent representation before classification. It enables Model B to adapt its representation to the dataset, but it may require stronger regularization (augmentation, dropout, early stopping) to generalize well when data are small.

3.2.4 Data Augmentation

Model B uses the transformation pipelines from torchvision only for the training data. No augmentation is performed for the validation and testing datasets.

- **Geometric augmentation:** Random rotations ($\pm 5°$ to $\pm 10°$), affine translations, optional flips
- **Photometric augmentation:** Brightness/contrast jitter, optional Gaussian noise

These are typically used to prevent overfitting by adding invariance to factors that are considered as "nuisance factors," such as position, small rotations, and intensity variation, while keeping the class label.

4 Implementation

Implementation of the above models will be described in detail in this section. The implementation would deal with the handling of the datasets, external libraries, and the detailing of code structure. The full code implementation can be obtained from the project repository on (GitHub: https://github.com/KamalJain777/AMLS_25_26_SN24205421).

4.1 Dataset Description and Handling

The experiments are performed on the **BreastMNIST** dataset: grayscale ultrasound images (28×28 pixels) with binary labels (benign/malignant). The official dataset provides data in NumPy archive format (.npz) [1]. The dataset consists of:

Table 1: BreastMNIST split sizes and class distribution.

Split	Total	Benign	Malignant
Train	546	147	399
Val	78	21	57
Test	156	42	114

Train/Validation/Test Protocol. The implementation follows the provided data splits:

- The training data is utilized for fitting the parameters of the model,
- The validation set is employed in model selection, such as early stopping, and in the tuning of hyperparameters when necessary,
- The data for the test split will not be used for selection purposes but will be held out for final reporting.

Preprocessing. All images are converted to float32 type and intensity-scaled from [0, 255] to [0, 1] range followed by model-specific normalization:

- **Model A (classical ML):** Normalization using the z-score transformation with the mean and std calculated from the training split only, to prevent leakage. For this particular run, the values for the training set were mean = 0.3276 and std = 0.2057 (scaled to [0, 1]).

- **Model B (deep learning):** Min-max scaling to the range [0, 1] is applied, as the data needs to be compatible with the image augmentation pipeline from PIL/torchvision.

Data Augmentation (optionally enabled) is applied to the training split only; validation and test datasets are never augmented.

4.2 External Libraries and Tools

The implementation leverages standard ML libraries:

- **NumPy** Array operations and loading .npz data.
- **SciPy** Classical (non-PyTorch) augmentation operations (rotate/shift/blur).
- **scikit-learn:** PCA, feature scaling, RBF SVM classifiers, cross-validated grid search
- **scikit-image:** HOG feature extraction
- **PyTorch/torchvision:** CNN modeling, training loops, and transform-based augmentation.
- **Matplotlib/Seaborn:** Plotting samples, class distributions, confusion matrices, ROC curves, and metric comparisons.
- **PyYAML** For loading YAML configuration files.
- **tqdm** Progress bars during training.

Execution entry point: All experiments are run using `main.py`, which handles command-line arguments to execute Model A, Model B, or both. Model configurations are defined in YAML files (`model_A/config.yaml`, `model_B/config.yaml`) with CLI can be used to provide common settings (e.g., training budget and output directory). Each run writes:

- Structured results to JSON/CSV (metrics, predictions/probabilities if available),
- Visualizations (samples, class distribution, confusion matrices, ROC curves),
- For Model B, per-epoch training history and learning-curve plots.

The runs reported in this document were executed using: `python main.py --model A --mode train --out_dir results`
`python main.py --model B --mode train --out_dir results`.

4.3 Model A Implementation

Code Organization:

- `model_A/dataset.py` — loads BreastMNIST.npz, returns train/val/test splits
- `model_A/preprocessing.py`: reusable preprocessing operations (scaling/normalization).
- `model_A/features.py` — FeatureExtractor class supporting raw, PCA, HOG modes
- `model_A/classifier.py` — ModelAClassifier implementing RBF SVM with GridSearchCV
- `model_A/trainer.py` — training and evaluation pipeline
- `model_A/augmentation.py` — NumPy/SciPy augmentation utilities for training-set augmentation.
- `model_A/evaluator.py` — metrics computation and result saving
- `model_A/visualizations.py`: plots samples and evaluation figures (confusion matrices, ROC, tables).

Feature pipelines: After preprocessing, one of three feature pipelines is used:

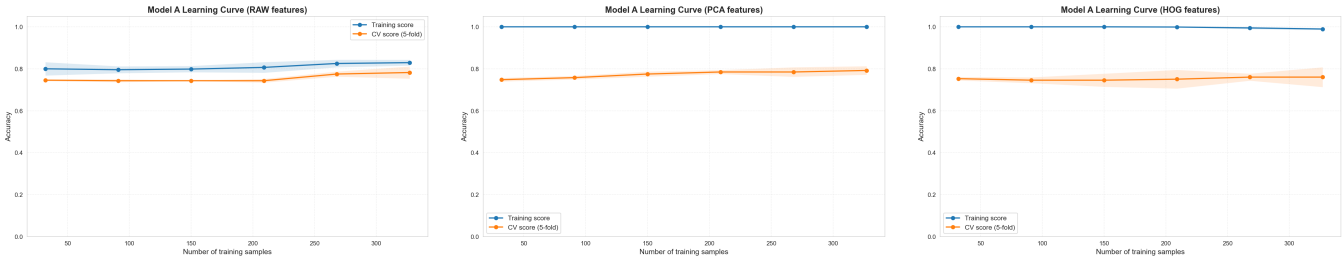
- **RAW**: Flatten $28 \times 28 \to 784$.
- **PCA**: Fit PCA on training data features and project to d components.
- **HOG**: Compute gradient-orientation histograms over cells/blocks.

Each pipeline outputs a 2D feature matrix of shape (N, D), which is then passed to the SVM.

Hyperparameter Selection (GridSearchCV). The SVM is using RBF kernel. Hyperparameters are chosen by cross-validated grid search over $C \in \{0.1, 1, 10, 100, 1000, 10000\}$ and $\gamma \in \{10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}\}$ using 5-fold cross-validation on the training set. It enables a consistent, automated tuning strategy across feature representations.

Model A "training curve" (learning curve): In contrast to neural networks, the training process for the SVM is not an iterated process over epochs, which would allow a meaningful curve of loss over epochs to be drawn. We instead employ the standard classical ML learning curve, which plots the performance on the training set and the cross-validation score as a function of the size of the training set.

Figure 3 shows training and CV accuracy increasing with dataset size, indicating the model is data-limited rather than capacity-limited. The persistent gap between training and CV curves suggests modest overfitting even with regularization, motivating the need for either more data or stronger feature engineering.



(a) RAW features (b) PCA features (c) HOG features

Figure 3: Model A learning curves: training and 5-fold CV accuracy vs training-set size for features representations

4.4 Model B Implementation

Code Organization:

- `model_B/dataset.py` — PyTorch Dataset
- `model_B/model.py` — ModelBNet defining ResNet-style neural network
- `model_B/trainer.py` — Training loop with early stopping and learning-rate scheduling
- `model_B/augmentation.py` — Torchvision transform pipeline
- `model_B/evaluator.py` — Evaluation on test set with metric computation
- `model_B/visualizations.py`: For platting learning curves, confusion matrix, ROC curve, and augmentation comparisons.

Learned feature pipeline: In Model B, the feature representation pipeline is learned end-to-end. The convolutional layers learn increasingly abstract representations, and the residual blocks ensure stable training through skip connections:

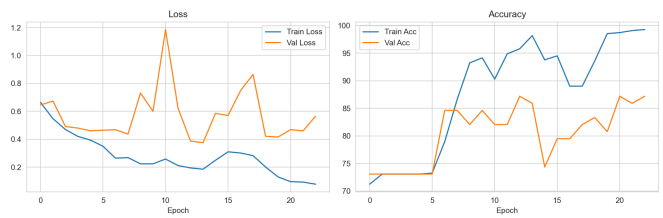
$$\mathbf{y} = \mathbf{x} + \mathcal{F}(\mathbf{x}, \theta)$$

Training pipeline, convergence, and stopping criteria: The training is performed using mini-batch optimization with cross-entropy loss and the Adam optimizer (learning rate=0.001, weight decay=10^{-4}, epochs = 50 with batch size 32).

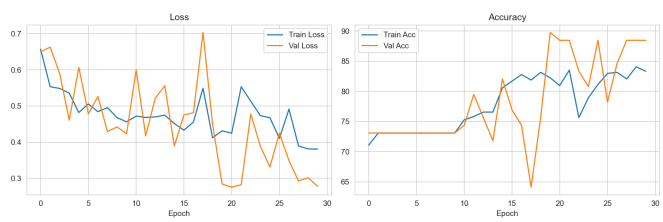
Convergence monitoring. During training, losses and/or accuracies are recorded per epoch in a history file. Learning curves are also plotted as shown in Figure 4. Learning curves are utilized to analyze convergence as well as analyze overfitting based on differences in losses (e.g., a widening gap between training and validation loss).

Stopping criterion (early stopping). In our code, we apply early stopping based on validation accuracy. If validation accuracy does not improve for a series of epochs, training is terminated. In our experiments, early stopping occurred at epoch number 23 for experiments without data augmentation and at epoch number 30 for experiments with data augmentation.

Evaluation and outputs: Model B is assessed on its performance on the held-out test data using the same evaluation metrics as Model A. Additionally, when turned on, it saves confusion matrices, ROC curves, and training curves. These outputs help in analyzing how model capacity and augmentation influence performance relative to Model A.



(a) Without augmentation showing overfitting with diverging train-validation gap



(b) With augmentation demonstrating improved regularization and better validation performance

Figure 4: Model B training curves comparison

5 Experimental Results and Analysis

5.1 Evaluation Metrics

Standard classification metrics reported on held-out test set, which is not used during training/validation :

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{ROC-AUC} = \text{Area under receiver operating characteristic curve}$$

5.2 Model A: Classical ML Results

Feature Pipeline Comparison (Table 2).

The best test accuracy of 79.49% and F1-score of 0.8750 for RAW features indicate that for such a number of samples and pre-processing, a highly non-linear classifier with less requirement for feature engineering can definitely compete. The F1-scores for PCA and HOG features were comparable but slightly lower. The high recall for all the features corresponds to the conservative strategy adopted by the classifier for classifying images into their respective categories of benign/malignant.

Table 2: Model A: Feature representation comparison on test set

Feature	Acc	Prec	Rec	F1	AUC
RAW	0.79	0.79	0.98	0.88	0.83
PCA	0.78	0.80	0.94	0.86	0.84
HOG	0.79	0.82	0.91	0.86	0.74

Augmentation Ablation (Model A).

Augmentation resulted in a slight change in performance (accuracy dropped from 78.21% to 77.56%), indicating that the learned representations and the regularization property of the SVM classifier already offer enough robustness.

Setting	Acc	Prec	Rec	F1	AUC
W/o aug	0.78	0.80	0.94	0.86	0.84
W/ aug	0.78	0.80	0.93	0.86	0.85

Table 3: Model A: Impact of augmentation (PCA features)

The results in the above table aligns with the literature on classical machine learning, where the benefits of augmentation may be limited for hand-designed features [3].

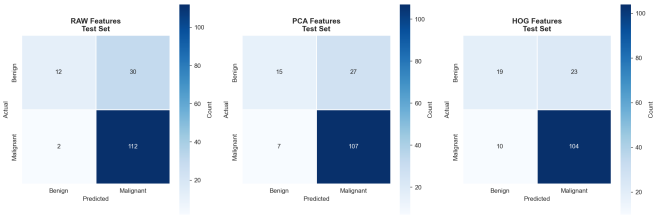


Figure 5: Model A: confusion matrices for RAW, PCA and HOG (test set)

- RAW features are a baseline and are highly dimensional (784) and may suffer from overfitting.
- PCA can reduce noise and dimensionality, and can potentially help in improving generalization.
- HOG concentrates on edges/texture and could be better at detecting structural patterns than individual pixels.

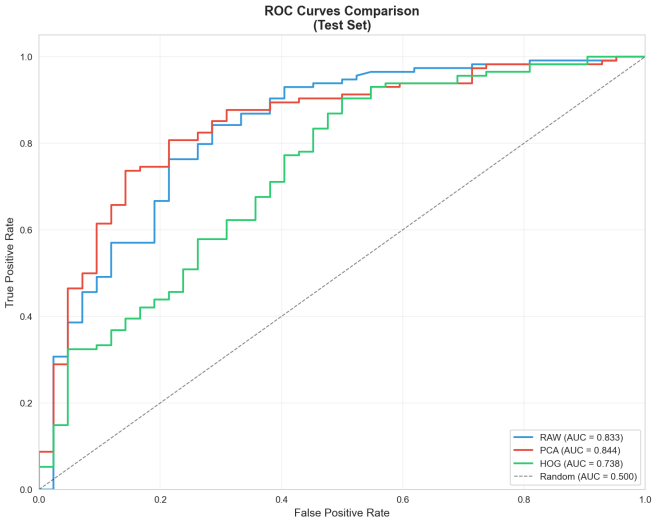


Figure 6: Model A: ROC curves for different feature representations (RAW, PCA, HOG) on test set

5.3 Model B: Deep Learning Results

Test-Set Performance. Model B achieved superior performance on the held-out test set:

Setting	Acc	Prec	Rec	F1	AUC
W/o aug	0.81	0.84	0.92	0.88	0.80
W/ aug	0.83	0.83	0.96	0.89	0.91

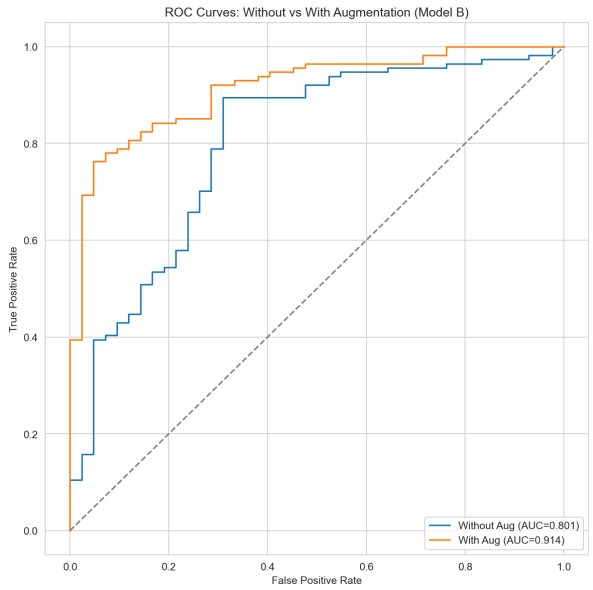
Table 4: Model B: Test set performance with and without augmentation

Best-case performance: Accuracy = 82.69%, F1-Score = 0.8907, ROC-AUC = 0.9142. Augmentation provided substantial regularization benefit, with AUC improving from 0.8005 to 0.9142—a 14.4% relative improvement. This demonstrates augmentation's role as a regularizer in neural networks to reduce overfitting [3, 4].

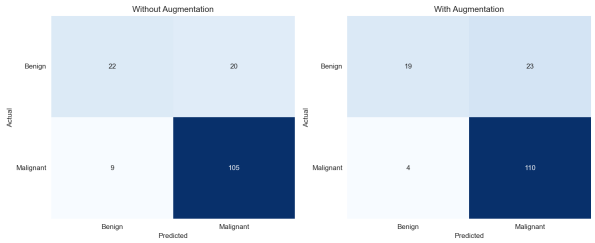
Training Convergence and Stopping Criterion. Early stopping was applied such that training would stop if validation performance was not improved within a given patience, around epoch 20-25. The learning curves show:

- Training accuracy increasing steadily to ∼98%
- Validation accuracy plateauing around 86-88% before stopping
- Validation loss showing characteristic overfitting behavior

This pattern confirms augmentation's regularization benefit: With augmentation, the validation curves stabilize at higher performance and AUC improves substantially [8].



(a) ROC curves showing substantial AUC improvement from 0.801 to 0.914 with augmentation



(b) Confusion matrices demonstrating better class separation with augmentation reducing false negatives from 9 to 4 while maintaining high true positive rate

Figure 7: Model B augmentation impact

Error Analysis Analysis of misclassified cases reveals both models performance impacts with: (i) low-contrast images where benign/malignant boundaries are ambiguous, (ii) images with significant artifacts or noise, and (iii) edge cases near the decision boundary. Model B shows better robustness to (i) and (ii) due to learned feature representations, while Model A's fixed features are more sensitive to image quality variations.

Summary: Data augmentation significantly improves Model B's discriminative ability, increasing ROC-AUC by 14.4% and reducing false negatives by 56% (9 to 4). The improved recall is particularly critical in breast tumor classification where missing malignant cases carries high clinical cost.

5.4 Model A vs Model B: Overall Comparison

Test-Set Metrics Comparison.

Key Findings:

Table 5: Overall comparison: Model A vs Model B on test set

Metric	A (PCA)	B (aug)	Improv.
Accuracy	0.78	0.83	+4.5%
Precision	0.80	0.83	+2.9%
Recall	0.94	0.96	+2.6%
F1-Score	0.86	0.89	+2.8%
ROC-AUC	0.84	0.91	+7.0%

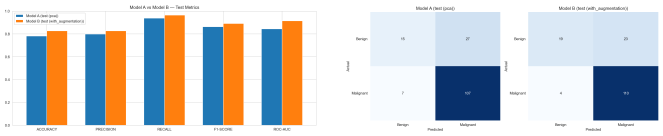


Figure 8: Model A vs Model B comparison on test set: (left) performance metrics showing Model B superiority across all measures with highest gains in ROC-AUC (+7.0%), and (right) confusion matrices demonstrating Model B's improved malignant detection with fewer false negatives (4 vs 7)

1. **Deep Learning Outperforms Classical ML:** Model B achieved higher test accuracy (82.69% vs 78.21%), substantially higher ROC-AUC (0.914 vs 0.844), and better overall discriminative ability.

2. **Augmentation Efficacy Differs by Paradigm:** Augmentation yielded negligible benefit for classical ML (+0.5% F1) but substantial benefit for deep learning (+2.8% F1, +7% AUC). This aligns with theoretical understanding: deep networks benefit from input diversity to enforce learned invariances, while hand-crafted features already encode robustness [3, 8].

3. **Model Complexity Trade-off:** Model B's end-to-end learning captures complex patterns but requires more regularization (augmentation, early stopping) to prevent overfitting. Model A provides interpretability via feature inspection but remains competitive only with careful feature engineering.

4. **Clinical Implications:** Both models show high recall (≥93%), critical for medical imaging where false negatives (missed malignancies) are costly. Model B's higher precision reduces false positives, improving clinical workflow [2, 4].

6 Conclusion

6.1 Summary of Findings

This report benchmarked classical machine learning approaches (radial basis SVM with hand-crafted features) with deep learning approaches (ResNet-like convolutional network) on BreastMNIST to understand the impact of capacity, features, and augmentation on medical image

classification tasks.

Model A (Classical ML) achieved 79.49% test accuracy using RBF SVM on hand-crafted features. Feature comparison revealed RAW pixels outperformed PCA and HOG, suggesting the RBF kernel's non-linearity compensates for minimal feature engineering. Augmentation provided marginal benefit, consistent with classical ML theory where pre-computed features already encode robustness [5, 7].

Model B (Deep Learning) achieved superior test accuracy (82.69%), F1-score (0.8907), and ROC-AUC (0.9142). Crucially, augmentation substantially improved performance (AUC +7%), demonstrating that learned representations benefit from input diversity to enforce invariances [3, 8]. Learning curves confirmed early stopping successfully prevented overfitting.

Key Trade-offs:

- Classical machine learning methods are interpretable, but require careful feature engineering.
- Deep learning methods have the advantage of increased representation power, but they depend on data augmentation and regularization.

6.2 Future Directions

Future Possible areas of future work are: (i) transfer learning on pre-trained ImageNet models on the BreastMNIST dataset, (ii) ensemble learning strategies that combine classical and deep learning methods, (iii) analysis of larger medical image datasets such as the MedMNIST benchmarks, and (iv) analysis of the model's predictive uncertainty. [2, 4].

References

[1] Yang, J., Shi, R., Wei, D., et al. (2021). MedMNIST v2: A large-scale lightweight benchmark for medical image classification. *Scientific Data*, 8, 295. https://doi.org/10.1038/s41597-021-00996-7

[2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning* (Adaptive Computation and Machine Learning Series). MIT Press.

[3] Murphy, K. P. (2012). *Machine Learning: A Probabilistic Perspective*. MIT Press.

[4] Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press.

[5] Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 886–893). IEEE. https://doi.org/10.1109/CVPR.2005.177

[6] Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer-Verlag.

[7] Schölkopf, B., & Smola, A. J. (2002). *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.

[8] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778). IEEE. https://doi.org/10.1109/CVPR.2016.90

[9] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)* (pp. 234–241). Springer. https://doi.org/10.1007/978-3-319-24574-4_28

[10] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324. https://doi.org/10.1109/5.726791

[11] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.